

CSE340: Computer Architecture

Assignment 3

Chapter 3

Note: For the questions below, if necessary, Consider 5 digits while performing the decimal to binary conversion of the floating portion.

Question 01

Suppose you want to set up a multiplication hardware following the long multiplication method that can multiply 0x82 with 0x85. Now answer the following questions:

- i. Size of the multiplicand register? = 16 bits
- ii. Size of the multiplier register? = 8 bits
- iii. Size of the product register? = 16 bits
- iv. Size of the ALU? = 16 bits
- v. How many iterations are needed to complete the multiplication? = 8 bits

Question 02

Suppose you want to set up a multiplication hardware following the optimized multiplication method that can multiply 0x82 with 0x85. Now answer the following questions:

- i. Size of the multiplicand register? = 8 bits
- ii. Size of the multiplier register? = No multiplier register needed; It is embedded in the product register
- iii. Size of the product register? = 16 bits
- iv. Size of the ALU? = 8 bits
- v. How many iteration is needed to complete the multiplication? = 8 bits

Question 03

Explain the significance of “control” in both long and optimized multiplication

hardware.

= In case of both long and optimized multiplication, there are multiple operations to perform such as left shifting/right shifting/ addition. Control decides the sequence of operations for both hardware setups . (When to perform which operation)

Question 04

mul x5, x6, x7;

What will be stored in register x5 after executing the given instruction?

= lower 64 bit of the product

Question 05

mulh x5, x6, x7;

Are the register sources here treated as signed in this instruction?

= Yes

Question 06

Suppose we want to multiply 8 with -9 and see the result of the upper 64 bits. Write the name of the instruction to do so.

= 8 is unsigned and -9 is signed. Mulhsu

Question 07

How does optimized multiplication improve efficiency and performance compared to traditional long multiplication, especially in terms of speed and computational complexity?

= Optimized multiplication improves efficiency by reducing register usage, minimizing data movement, and lowering the number of sequential operations in the datapath.

Question 08

	Instruction name
1	mul
2	mulh

3	mulhu
4	mulhsu

You **must** answer the following questions for the explanation of each of the instructions mentioned above:

(i) Explain their syntax.

(iii) Write an example for each of the instructions.

= Check slide and do your own research

Question 09

How do we store a floating point number in a register?

= We convert the floating point number into an IEEE-754 floating point format and then store than value into a floating point register.

Question 10

What is the significance of IEEE754 floating point format?

= You can get the answer from the previous question.

Question 11

Compare IEEE754 single precision vs double precision floating point format systems.

	Sign Bit (bit)	Exponent (bit)	Fraction (bit)
Single Precision	1	8	23
Double Precision	1	11	52

Double precision gives you a larger range than single precision.

Consider a number, where fraction field = 1111 0101 1010 0000 0000 0000 1010

And the biased exponent field = 1010

So, this number can be easily represented using single precision format.

However, the fraction field will be limited to 23 bits but the mentioned number has 28 bits in it.

So, when we store this number using single precision format, we compromise with precision, using double precision would help with maintaining the precision.

Question 12

Why is a bias added to the actual exponent in the IEEE 754 floating-point representation, and how does this affect the encoding of both positive and negative exponents?

= Exponent can be a positive or negative number.

So, while storing the exponent field, we also have to store its sign bit.

In order to get rid of this sign bit we add bias with exponent, make it unsigned and then store the biased exponent.

Question 13

Explain the significance of the following fields:

- i. Fraction. = How precisely can you represent the number
- ii. Exponent. = How large or small number can you represent

Question 14

a. Smallest positive value in single precision format.

Sign Bit (1 bit)	Exponent (8 bits)	Fraction (23 bits)
0	0000 0001	0000 0000 0000 0000 000

$$\text{Bias} = (2^{8-1}) - 1 = 127$$

$$\text{Actual Exponent} = 1 - 127 = -126$$

$$\begin{aligned} & (-1)^0 \times (1 + 0.0000\ 0000\ 0000\ 0000\ 000) \times 2^{-126} \\ &= 1.0000\ 0000\ 0000\ 0000\ 000 \times 2^{-126} = 1.175 \times 10^{-38} \end{aligned}$$

b. Largest positive value in single precision format.

Sign Bit (1 bit)	Exponent (8 bits)	Fraction (23 bits)
0	1111 1110	1111 1111 1111 1111 111

$$\text{Bias} = (2^{8-1}) - 1 = 127$$

$$\text{Actual Exponent} = 254 - 127 = 127$$

$$(-1)^0 \times (1 + 0.1111\ 1111\ 1111\ 1111\ 111) \times 2^{127}$$

$$= 1.1111\ 1111\ 1111\ 1111\ 111 \times 2^{127} = 3.402 \times 10^{38}$$

c. Smallest negative value in single precision format.

Sign Bit (1 bit)	Exponent (8 bits)	Fraction (23 bits)
1	1111 1110	1111 1111 1111 1111 111

$$\text{Bis} = (2^{8-1}) - 1 = 127$$

$$\text{Actual Exponent} = 254 - 127 = 127$$

$$(-1)^1 \times (1 + 0.1111\ 1111\ 1111\ 1111\ 111) \times 2^{127}$$

$$= -1.1111\ 1111\ 1111\ 1111\ 111 \times 2^{127} = -3.377 \times 10^{38}$$

d. Largest negative value in single precision format.

Sign Bit (1 bit)	Exponent (8 bits)	Fraction (23 bits)
1	0000 0001	0000 0000 0000 0000 000

$$\text{Bis} = (2^{8-1}) - 1 = 127$$

$$\text{Actual Exponent} = 1 - 127 = -126$$

$$(-1)^1 \times (1 + 0.0000\ 0000\ 0000\ 0000\ 000) \times 2^{-126}$$

$$= -1.0000\ 0000\ 0000\ 0000\ 000 \times 2^{-126} = -1.175 \times 10^{-38}$$

Question 15

Subtract -4.0210 from 28.4810 using IEEE-754 single-precision floating-point representation. Check if the result has overflow or underflow or none.

Note: Consider 5 digits while performing the decimal to binary conversion of the floating portion.

$$= 28.4810 - (-4.0210) = 28.4810 + 4.0210$$

$$28.4810 = 11100.01111 = 1.1100\ 01111\ 1 \times 2^4$$

$$4.0210 = 100.00000 = 1.0000\ 000\ 1 \times 2^2$$

$$1.1100\ 01111\ 1 \times 2^4 + 1.0000\ 000\ 1 \times 2^2$$

$$= 1.1100\ 01111\ 1 \times 2^4 + 0.0100\ 0000 \times 2^4$$

$$= 10.0000\ 0111\ 1 \times 2^4$$

$$= 1.00000\ 0111\ 1 \times 2^5$$

$$\text{Actual Exponent} = 5$$

$$\text{Bias} = (2^{8-1}) - 1 = 127$$

$$\text{Biased Exponent} = 5 + 127 = 132$$

$$\begin{aligned} \text{Biased Exponent Range} &= 0 \text{ to } 2^8 - 1 = 0 \text{ to } 255 \text{ [0 and 255 reserved]} \\ &= 1 \text{ to } 254 \end{aligned}$$

$1 < 132 < 254$; Therefore, No overflow or underflow.

Question 16

Given two numbers $X = 0x7ABCD$ and $Y = 0x1CDE3$ in IEEE-754 floating-point representation where 4-bits are assigned for the exponent, the rest of the bits are used for the sign and fraction fields. Perform $X+Y$ and show the decimal equivalent of your result.

Note: Consider 5 digits after the floating point while converting from decimal to binary or vice versa. You must show all the steps associated with the conversion and calculation.

=

$X = 0x7ABCD$; this is already in IEEE format. So, first find their corresponding number.

Same goes for Y

$$X = 0x7ABCD = 0111\ 1010\ 1011\ 1100\ 1101$$

So, Sign bit = 0

$$\text{Exponent bit} = 1111 = 15$$

$$\text{Fraction bit} = 010\ 1011\ 1100\ 1101$$

$$X = (-1)^{\text{sign bit}} \times (1 + \text{Fraction}) \times 2^{\text{Actual Exponent}}$$

$$= (-1)^0 \times (1 + 0.010\ 1011\ 1100\ 1101) \times 2^?$$

$$\text{Bias} = 2^{4-1} - 1 = 7$$

$$\text{Actual Exponent} = \text{Biased exp} - \text{Bias} = 15 - 7 = 8$$

$$= (-1)^0 \times (1 + 0.010\ 1011\ 1100\ 1101) \times 2^8$$

$$= 1.010\ 1011\ 1100\ 1101 \times 2^8$$

$$Y = 0x1CDE3 = 0001\ 1100\ 1101\ 1110\ 0011$$

So, Sign bit = 0

$$\text{Exponent bit} = 0011 = 3$$

$$\text{Fraction bit} = 100\ 1101\ 1110\ 0011$$

$$Y = (-1)^{\text{sign bit}} \times (1 + \text{Fraction}) \times 2^{\text{Actual Exponent}}$$

$$= (-1)^0 \times (1 + 0.100\ 1101\ 1110\ 0011) \times 2^?$$

$$\text{Bias} = 2^{4-1} - 1 = 7$$

$$\text{Actual Exponent} = \text{Biased exp} - \text{Bias} = 3 - 7 = -4$$

$$= (-1)^0 \times (1 + 0.100\ 1101\ 1110\ 0011) \times 2^{-4}$$

$$= 1.100\ 1101\ 1110\ 0011 \times 2^{-4}$$

$X + Y$

$$= 1.010\ 1011\ 1100\ 1101 \times 2^8 + 1.100\ 1101\ 1110\ 0011 \times 2^{-4}$$

$$= 1.0101\ 0111\ 1001\ 101 \times 2^8 + 0.0000\ 0000\ 0001\ 1001\ 1011\ 1100\ 011 \times 2^8$$

$$= 1.01010111101101 \times 2^8$$

$$= 1.34259 \times 2^8$$

$$= 3.4370304 \times 10^2$$

Question 17

file.d x5, f3, f5

(i) Why x5 is an integer register?

(ii) f3 and f5 both are single precision numbers. True or False. If false, write the correct statement.

(iii) after running the instruction you see that x5 register contains 0. What can you comment on?

(i) File compares two floating point registers' data. If comparison is true x5 will have 1 else 0.

1 and 0 are integer, therefore the integer register.

(ii) False. File.d -> .d means double precision

(iii) f3 and f5 does not have same data stored.

Question 18

Write the necessary RISC-V code to compare two floating point registers f1 and f2. If f1 and f2 are equal jump to a label called “**jumpEqual**” else jump to another label called “**jumpNotEqual**”

Note: both the numbers stored in f1 and f2 registers are double-precision floating point numbers.

```
fld x5, f1, f2
beq x5, x0, jumpNotEqual
bne x5, x0, jumpEqual
```

```
jumpNotEqual:
Beq x0, x0, exit
```

```
jumpEqual:
```

```
exit:
```

Question 19

Write the necessary RISC-V code to compare two floating point registers f1 and f2. If f1 is greater than f2, jump to a label called “**jumpGreater**” else jump to another label called “**jumpNotGreater**” and divide the variable c with 16 and store that answer in c.

Note: both the numbers stored in f1 and f2 registers are double-precision floating point numbers and variable c is stored in x19.

```
f1>f2 - jumpGreater; f2<f1
f1<f2 - jumpNotGreater
```

```
flt.d x5, f2, f1
beq x5, x0, jumpNotGreater
bne x5, x0, jumpGreater
```

```
jumpGreater:  
beq x0, x0, exit
```

```
jumpNotGreater:  
srli x19, x19, 4
```

Exit:

Question 20

```
if (a > b && a == b):  
    c = (a * b) + d  
else:  
    c = - ((a * b) + d)
```

Convert the given code into corresponding, RISC-V code. Assume, variables a, b, c and d are double precision floating point numbers and they are mapped to floating point registers f4, f5, f6 and f7.

Note:

(a * b) + d; use a single instruction to perform these operations.
- ((a * b) + d); use a single instruction to perform these operations.

```
flt.d x5, f5, f4  
addi x8, x0, 1  
beq x5, x8, secondConditionForIf  
runElse:  
fnmadd.d f6, f4, f5, f7  
beq x0, x0, exit
```

```
secondConditionForIf:  
feq.d x6, f5, f4  
addi x8, x0, 1  
beq x6, x8, runIf
```

```
beq x0, x0, runElse
```

```
runIf:
```

```
fmadd.d f6, f4, f5, f7
```

```
Exit:
```

Question 21

Two double-precision numbers a and b are stored consecutively in memory. The memory location of the second number, b is directly stored in register x19.

Convert the given high level code to corresponding risc-v code:

$res = (a - b)^2$

Note: res is mapped to f16

```
fld f2, 0(x19) # second number - b
```

```
addi x20, x19, -8
```

```
fld f3, 0(x20) # first number - a
```

```
fsub.d f1, f3, f2
```

```
fmul.d f16, f1, f1
```